

A Self-Learning, Self-Diagnosing Humidity Controller for a Low-Cost IoT Terrarium: Evaluation on a Real Enclosure Against Hysteresis Control

1st Kishor Tarafder

Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54520-3@student.aiub.edu

2nd MD. Shakhawat Hossen

Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54483-3@student.aiub.edu

3rd Tahomina Era

Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-55027-3@student.aiub.edu

4th Avishek Saha

Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54508-3@student.aiub.edu

5th Jannatun Nur Deba

Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54545-3@student.aiub.edu

6th MD. Alfaz Uddin

Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-55557-3@student.aiub.edu

Abstract—Low-cost terrarium and small-greenhouse controllers keep humidity with a fixed hysteresis rule that neither adapts to the enclosure nor notices when a part fails. This paper presents a controller in which one small model of the enclosure, six weights fitted by recursive least squares in absolute humidity, is used for three purposes at once: it is learned from the enclosure's own response, it plans the length of each mist burst, and it detects and names faults by comparing what should happen with what does. The model is physics-shaped, so each weight corresponds to a part (mister strength, leakage, fan, background), and a suspect-then-test procedure separates a leak from a dead mister by a short quiet watch followed by a mist test. The controller was evaluated on a real acrylic enclosure with an ultrasonic mist maker, two BME280 sensors and an ESP32, against the enclosure's existing 75 to 90 percent hysteresis rule, with all readings logged at 1 Hz. A three-round step test characterised the enclosure (mister ceiling 89.5 plus or minus 0.4 percent RH reached in 104 plus or minus 7 s). In one hour of alternating 30 min blocks on the same enclosure the learning controller held the target band 100 percent of the time with no misting, where the hysteresis rule held it 99.4 percent of the time using 14.6 min of mist. With the lid removed as an injected fault, the controller flagged an anomaly within 1 s, kept the band with three 35 s test bursts, and named the fault as a leak after 400 s; a later false alarm caused by a drying sensor was tested and cleared by the controller itself in 6.5 min. The model also re-fitted itself to the leakier, opened enclosure within 25 min without any misting data. Limitations are stated: the controller executed on a laptop over USB at 5 s steps rather than on the ESP32, the evaluation covers three hours on one enclosure without plants, and burst planning was exercised on the real enclosure only during the fault.

Keywords—adaptive control, fault detection, recursive least squares, ESP32, humidity, terrarium, TinyML

I. INTRODUCTION

Tropical terrarium plants such as Fittonia need relative humidity held in a narrow band, typically 75 to 90 percent. Hobby and low-cost commercial controllers achieve this with an on/off rule: mist when the reading falls below a lower threshold, stop at an upper threshold or after a fixed burst. The rule is simple and robust, but it has two weaknesses that this paper addresses. First, it does not know the enclosure. Its burst length and thresholds are fixed by the installer, so it over-mists a tight enclosure and under-mists a leaky one, and it cannot follow slow changes such as plant growth, a warming room or a wearing mister disc. Second, it does not know whether its actions work. A dead mister, an empty reservoir, an open lid or a stuck sensor all produce the same behaviour, misting into an enclosure that does not respond, until a person notices.

The contribution is a controller in which one small model does both jobs. The model predicts the next change in absolute humidity from the mist command, the fan state and the current humidity, using six weights that each correspond to a physical part of the enclosure. It is fitted on the fly by recursive least squares (RLS) with two forgetting rates, a fast learner that tracks the present and a slow learner that remembers the healthy enclosure. The same model plans each mist burst by simulating candidate lengths a few minutes ahead, and it detects faults by comparing prediction with measurement, then runs a short active test to name the fault. The state is six weights and two 6 by 6 matrices, about 2 kB, and each 5 s step needs a few hundred multiplications, so the design targets the ESP32 already present in the enclosure.

Three literature searches (Consensus Deep Search, September 2026) found on-device learning on microcontrollers for images, audio, gestures, solar forecasting and irrigation, but not for humidity control; model-predictive or learning-based humidity control exists in simulation, on PCs and on PLCs, but not on a low-cost microcontroller driving an ultrasonic humidifier; and no study was found that uses one learned model for both control and actuator fault diagnosis at the edge, or that measures climate outcome against the plain hysteresis rule on the same hardware. This paper reports such a measurement on a real enclosure, together with the enclosure characterisation and an injected-fault test, and states plainly what the three hours of data do and do not support.

II. RELATED WORK

Small models on microcontrollers for greenhouse climate are established. Codeluppi et al. forecast greenhouse air temperature on an edge device with an RMSE of 0.29 to 0.40 degrees C [1]; Ihoume et al. mapped sensor readings to five actuator actions with a 151-parameter network at 96 percent label accuracy [2]; Sukowati et al. replaced thresholds with a k-nearest-neighbour classifier on an ESP32 for a mushroom house including a mist maker [3]. These models are trained once on a computer and frozen, and they are scored on label accuracy rather than on climate outcome. On-device training itself exists in TinyOL [4], in the concept-drift-aware TinyML of Disabato and Roveri [5], which names sensor and actuator faults as one cause of drift but tests on image and audio data, and in TyBox [6], which generates incremental-learning code for classification. Predictive and learned control of greenhouse humidity has been studied in simulation [7], and a grey-box humidity model identified automatically on an STM32 was used to schedule a room humidifier without plants or fault handling [8]. Bicski et al. warn that a simple autoregressive model beat an autoencoder for anomaly detection on industrial time series [9], which motivates the linear model used here and the comparison against the plain rule rather than against a larger network. Adaptive control of this form is decades old; the contribution claimed is not the algorithm but its use, on a real low-cost enclosure, as a single model for control and diagnosis, evaluated against the controller such enclosures actually run.

III. SYSTEM

A. Enclosure and Hardware

The enclosure is an acrylic box of about 28 by 23 by 8 cm with a removable lid. Humidity is raised by one 5 V ultrasonic mist maker (DIY-4-1) standing in a water tray inside the box, switched through an opto-isolated MOSFET channel driven active-low from an ESP32 DevKit V1 GPIO. Two Bosch BME280 sensors on one I2C bus measure temperature, relative humidity and pressure: the control sensor at address 0x76 mounted high inside the box under a small droplet shield, and a reference sensor at 0x77 (SDO tied to 3.3 V) on the bench about 30 cm from the box. The lid was propped open on one edge to give the enclosure a drying path, by 1 cm in the fault session and 4 cm in the alternating session, since a closed enclosure in a 75 percent RH room loses under 2

percent RH in 20 min and gives a controller nothing to do. No plants were present during the runs reported here.

B. Firmware

A purpose-written laboratory firmware runs on the ESP32. It samples both sensors every second in forced mode with 1 times oversampling and streams temperature, humidity to 0.01 percent and pressure over USB, together with the mist commands, the control mode and sensor validity flags. It accepts commands to set the mode (off, manual, or the hysteresis rule), to switch either mist channel in manual mode, to change thresholds, and to write a marker line into the log. Readings are rejected unless temperature, pressure and the step from the previous sample are physically plausible, because a marginal bus was found to return values such as 100.00 percent RH or 0 degrees C that pass naive checks. A lost sensor is re-probed every 5 s, a 15 s hardware watchdog restarts the chip if it stalls, and mode and thresholds are restored from flash after a restart with the mist off. The hysteresis rule implemented in this firmware is the one the enclosure's production firmware uses: mist when RH falls below 75 percent, stop at 90 percent or after 300 s, then rest 180 s.

C. The Learned Model

Humidity is modelled in absolute terms, as vapour density in g per cubic metre computed from RH and temperature with the Magnus formula, which removes most of the effect of temperature on the reading. With DT equal to 5 s, the predicted change over one step is the inner product of six weights with the feature vector $[g_{m1}, g_{m2}, a, f_a, f, 1]$, where $m1$ and $m2$ are the mist command filtered with 5 s and 30 s time constants, g equals $\max(0.02, 1 - \text{RH}/100)$ accounts for slower evaporation of droplets near saturation, a is the absolute humidity above a 20 g per cubic metre reference, and f is the fan state. The first two weights are the mister's strength, the third the enclosure's leakage, the fourth and fifth the fan, and the sixth the background gain from plants, the water surface and the room. After every reading the weights are updated by RLS. Two learners run in parallel: a fast one with forgetting factor 0.998 (about 40 min of memory) and a slow one with 0.9998 (about 7 h). The fast learner is anchored to the slow one by a small pull each step, because closed-loop data does not identify all directions and an unanchored fast learner was found to explain the data with absurd cancelling weights. Prediction errors are clipped before learning, the covariance matrix is kept symmetric, floored and capped, and the weights are constrained to be physically sensible: the mister can only add water, the enclosure and the fan can only remove it.

D. Burst Planning

When the control sensor falls below 77 percent RH, the controller simulates candidate bursts of 5 s to 300 s ten minutes ahead with the slow model and chooses, by bisection, the shortest burst whose predicted peak reaches 88 percent. It then rests 60 s. During a declared fault the fast model is used and the target is lowered to 84 percent. On healthy days the controller records the start times and lengths of its bursts; if the sensor is later lost or stuck, it replays the median burst and gap of that rhythm rather than misting blind on a model that is only trusted for ten minutes.

E. Fault Detection and Self-Repair

Faults are detected in three layers. A missing reading, or twelve identical readings in a row, marks the sensor lost or stuck. After every two minutes of misting, and after every fan run, a part check asks whether the mister added, or the fan removed, the water the slow model expects, after subtracting the background drain measured while the enclosure was quiet, so that a leak cannot make the mister look dead; two failed checks in a row declare a fault. A cumulative-sum (CUSUM) test on the normalised prediction error detects humidity falling faster than the healthy model allows and raises a suspect-loss state. A plain error alarm cannot tell a leak from a dead mister, since both look like humidity falling too fast, so the suspect state triggers an active test: the controller stops misting for 4 min and watches, then mists and watches again. A leak drains in both phases; a dead mister fails only while misting; an enclosure that behaves normally in both phases is a false alarm and the state is cleared. During any fault the slow learner freezes so that a broken part is never learned as normal. On a dead main mister the controller switches to the second channel; on a leak or a stalled fan it raises an alarm and keeps controlling with the fast model. The slow weights are snapshotted every 6 h when they predict at least as well as the previous snapshot, and restored at once if they change without learning or predict twice as badly as the snapshot for 30 min. A maintenance warning is issued when the mister falls below 80 percent of its day-one strength.

F. Execution Platform in This Study

In the experiments reported here the model, planner and fault logic ran as a Python program on a laptop connected to the ESP32 over USB. Every 5 s it read the latest control-sensor sample from the firmware's 1 Hz stream, executed one step of the controller, and sent the resulting mist command back. The firmware executed the command with its own safety limits. The program is written with fixed-size arrays and plain arithmetic for a line-by-line port to C; that port was not completed for this study and is discussed in Section VI. The model was seeded before the runs by an ordinary least-squares fit of the same six features to the step-test data of Section IV-A, so the controller acted from its first minute and only fine-tuned thereafter.

IV. EXPERIMENTAL METHOD

A. Enclosure Characterisation

With the lid closed, a scripted protocol ran three rounds of: 5 min quiet, mist on until the control reading stopped rising (less than 0.3 percent over 60 s, after at least 90 s), then 20 min off. The mister's dry-air strength was taken from the absolute-humidity slope over the first 60 s of each burst divided by g at the start, and the plateau, the time to it, the drop over the 20 min off period and the temperature change were recorded.

B. Injected Faults

With the lid propped 1 cm, the hysteresis rule ran for 15 min and the learning controller for 47 min. During the learning controller's run the lid was removed completely at a marked time and replaced after 8.6 min; later a wet tissue was held on the control sensor for 3.7 min and removed. The time

from each marker to the controller's fault declarations was measured from the log.

C. Alternating Comparison

With the lid propped 4 cm, the wiring redone and both sensors verified at 100 percent valid readings over 2 min, the two controllers alternated in 30 min blocks: rule, learning, rule, learning, on the same enclosure in one uninterrupted two-hour night session, each block starting from the state the previous one left. Metrics per block are the fraction of valid control-sensor readings inside the 75 to 90 percent band, above 90 and below 75 percent, mist minutes, and the number of mist switch-ons. Mist minutes are used as the water measure; the water volume was not weighed.

D. Data Quality

All analysis uses only samples flagged valid by the firmware filter. The fraction of valid samples is reported for every run. Session logs, the controller's internal log of weights, error and fault state at every 5 s step, and the analysis scripts are kept with the data.

V. RESULTS

A. Enclosure Characterisation

Table I and Fig. 1 give the three rounds; all 4825 samples were valid. The mister raises the control reading to a ceiling of 89.5 plus or minus 0.4 percent RH in 104 plus or minus 7 s and cannot exceed it; longer bursts add droplets to surfaces, not vapour to the air. The dry-air strength was 0.098 plus or minus 0.012 g per cubic metre per second. The closed enclosure lost only 1.7 plus or minus 0.4 percent RH in 20 min with the room at 75 percent, too little to fit a decay constant; an earlier run the same day with one end of the box open lost 5 percent in 11 min. The air warmed 0.2 to 0.4 degrees C while the disc ran and then cooled about 0.5 degrees C below the start as the fog evaporated. Two practical consequences follow: the production firmware's 300 s burst cap is three times longer than useful for this enclosure, and its 90 percent upper threshold is never reached, so every rule burst runs to the cap.

TABLE I. STEP TEST, LID CLOSED, ONE MIST MAKER (25 SEP 2026)

Round	Start %RH	Peak %RH	Mist s	Strength g/m ³ /s	Drop 20 min
1	83.0	89.0	110	0.086	1.8 %
2	86.9	89.5	97	0.110	2.0 %
3	87.1	89.9	104	0.099	1.2 %
mean $\pm$ sd		89.5 $\pm$ 0.4	104 $\pm$ 7	0.098 $\pm$ 0.012	1.7 $\pm$ 0.4 %

Step test 2026-09-25: one mist maker, measured at 1 Hz (blue = mist on)

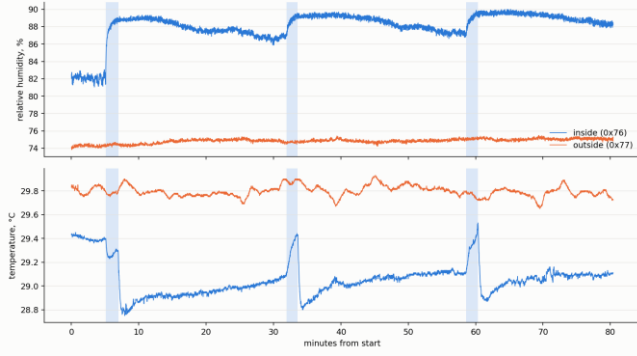


Fig. 1. Step test: inside and outside humidity (top) and temperature (bottom) over three mist rounds; shaded bands mark mist on.

B. Injected Faults

Table II summarises the blocks of the fault session and Table III the controller's responses; Fig. 2 shows the traces. The hysteresis rule's single burst ran the full 300 s cap and took the enclosure from 70.8 to 88.2 percent. Before the fault the learning controller correctly did nothing for 25 min, since the rule had left the enclosure at 87 percent and the 1 cm gap drained it only slowly; in that time its slow leak weight moved from minus 0.036 to minus 0.131 per step and the background weight from 0.19 to 1.00, a re-fit of the opened enclosure made without any misting data. When the lid was removed the CUSUM raised suspect-loss within 1 s of the marker. The controller ran its 4 min quiet watch, then three test bursts of about 35 s at 60 s intervals, keeping the enclosure inside the band, and declared lid open / leak 400 s after the marker. The wet tissue did not latch the sensor, but it locked the I2C bus; the firmware watchdog restarted the chip within 15 s and both sensors were re-found in 1 s, while the laptop-side controller, which at that time did not handle a chip restart, had to be relaunched (six seconds of control lost; the handling was added afterwards). When the tissue was removed the reading fell from 90 to 74 percent in 40 s as the sensor dried; the controller raised suspect-loss again, ran its quiet-then-mist test, and cleared the alarm as a false alarm 6.5 min later, having found the enclosure behaving normally in both phases.

TABLE II. FAULT SESSION BLOCKS, LID 1 CM (26 SEP 2026)

Block	min	In 75–90 %	Mist min	Bursts
Hysteresis rule	15.1	96.5 %	5.0	1
Learning, incl. lid-off fault	37.1	95.4 %	1.1	3
Learning, wet-sensor test	10.1	59.3 %	1.5	1

TABLE III. INJECTED FAULTS AND THE CONTROLLER'S RESPONSE

Event (time)	Response
Lid fully off (23:02:11)	suspect-loss after 1 s; quiet watch 4 min; three 35 s test bursts; “lid open / leak” after 400 s; band held meanwhile
Lid back, 1 cm (23:10:47)	recovered; fault cleared once the enclosure matched the healthy model for 10 min
Wet tissue on sensor (23:12:50)	sensor did not stick; I2C bus locked; watchdog restart in 15 s, sensors re-found in 1 s
Tissue removed (23:16:30)	reading fell 90 to 74 % in 40 s; suspect-loss raised; tested; cleared as false alarm after 6.5 min

Lab session 2026-09-26: old rule vs brain on the real box, with injected faults (blue = mist on)

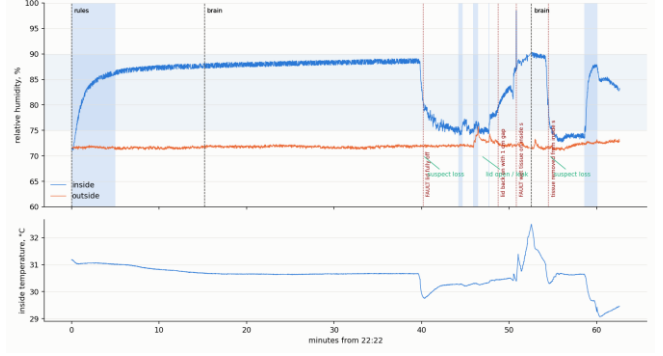


Fig. 2. Fault session: humidity (top) and inside temperature (bottom); dotted red lines mark injected events, green labels the controller's verdicts.

C. Alternating Comparison

Table IV gives the four blocks; every sample in all four was valid. Fig. 3 shows the traces. In each rule block the enclosure fell to 75 percent, the rule misted for the full 300 s cap to 87 to 88 percent, and the enclosure drained back in about 10 min; over its hour the rule misted 14.6 min in four bursts and held the band 99.4 percent of the time, the remainder being the dips below 75 percent that trigger it. In both learning blocks the enclosure, left alone, settled between 77 and 82 percent RH: with the 4 cm gap and the water tray, the enclosure's own equilibrium lies inside the band. The learning controller's planner, which acts only below 77 percent, was never triggered; it held the band 100 percent of the time with zero misting, and its slow model moved again (leak weight minus 0.036 to minus 0.093 per step over the two blocks). The blocks alternated, and each learning block inherited the enclosure at 75 to 88 percent from the preceding rule block; the reference sensor drifted from 73 to 71 percent over the two hours, so conditions were slightly drier for the later blocks. One interrupt-watchdog restart of the ESP32 occurred at 02:50 in the last block; the controller detected the restart from the uptime counter, re-armed manual mode and continued with its memory intact, losing six seconds.

TABLE IV. ALTERNATING 30 MIN BLOCKS, LID 4 CM (27 SEP 2026)

Block	In 75–90 %	Below 75 %	Mist min	Bursts	%RH range
Rule, 00:55	98.9 %	1.1 %	10.2	3	72–87
Learning, 01:25	100.0 %	0.0 %	0.0	0	77–82
Rule, 01:55	99.9 %	0.1 %	4.4	1	75–88
Learning, 02:25	100.0 %	0.0 %	0.0	0	79–88
Rule, 60 min	99.4 %	0.6 %	14.6	4	
Learning, 60 min	100.0 %	0.0 %	0.0	0	

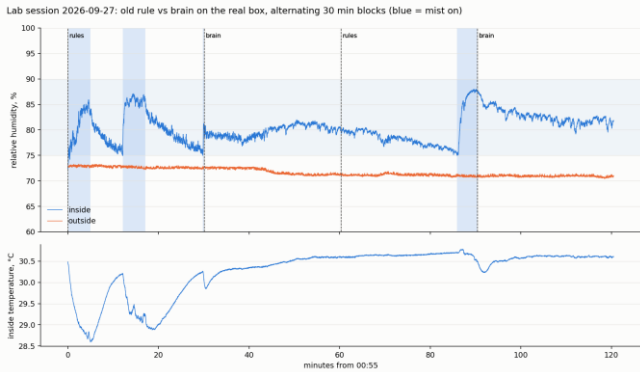


Fig. 3. Alternating session: the rule mists to its 300 s cap whenever the enclosure reaches 75 %; the learning controller, judging the enclosure's own equilibrium sufficient, never mists.

D. Simulation as a Logic Check

Before the hardware runs the controller was exercised on a simulated enclosure for 76 h with slow disc wear, a memory corruption and four injected faults, with the simulator's constants later replaced by the measured values of Section V-A. In that setting it held the band 99.2 percent of healthy hours against 93.1 percent for the rule, used 20 percent less mist time, named every fault correctly on seven random seeds without false alarms, and rolled back its corrupted weights in the same step. The simulation is reported only as evidence that the logic behaves as designed; it is not a result about the real enclosure.

VI. DISCUSSION AND LIMITATIONS

The measured results support four claims. A six-weight model seeded from a 90 min step test and updated on the fly can control a real enclosure at least as well as the hysteresis rule it replaces, holding the band 100 percent of the time in the alternating hour against 99.4 percent, and 95.4 percent against 96.5 percent in the fault session where the learning controller also carried an injected fault. It does so with far less mist time: none against 14.6 min per hour, and 1.1 min against 5.0 min in the fault-session blocks. It detects an unexplained loss within a second, keeps control while it investigates, names a leak correctly, and withdraws its own false alarm by testing rather than by timeout. And it follows a changed enclosure, re-fitting its leak weight by a factor of 3.6 within 25 min of the lid being propped, without any misting data. The engineering lessons are also results: a wet BME280 can lock the I2C bus and be recovered by a hardware watchdog without a host restart; corrupted bus reads pass naive range checks; and a sensor that reports a fixed 179.4 degrees C and 100 percent RH is not damaged but sitting at its power-on defaults because register writes are failing, which a solid clock line fixes and a power cycle does not.

The limitations are stated plainly. The controller executed on a laptop over USB at 5 s steps, not on the ESP32; the code is written for a direct port and its state is about 2 kB, but the on-chip run is future work. The evaluation is three hours on one enclosure in one room, at night, without plants, and mist minutes stand in for water volume. The zero-misting result in the alternating session is a result about an enclosure that barely needed misting, so it shows the learning controller recognising that fact rather than planning bursts; the only

real-enclosure bursts it planned were the three 35 s test bursts during the lid fault, where it held the band. A drier room or a scheduled fan would make the planner act and would test it properly. The dead-mister fault and the backup switchover were exercised only in simulation. The false alarm after the tissue was removed was self-cleared, but it would be prevented outright by checking the sensor's own temperature, which fell 2 degrees C as it dried; that check is a natural addition. Finally, the interrupt-watchdog restart of the ESP32 in the last block is unexplained and must be root-caused before an on-chip deployment.

VII. CONCLUSION AND FUTURE WORK

One small learned model, updated on the fly, can serve a low-cost terrarium as its humidity planner and its fault detector at once. On a real enclosure it matched or beat the existing hysteresis rule on time in band while misting far less, named an injected leak, cleared its own false alarm, and re-fitted itself to a changed enclosure. The next steps are to port the controller to the ESP32 and repeat the alternating comparison on the chip, to run it in conditions where the enclosure needs misting so that the planner is tested, to inject the dead-mister fault with the backup channel connected, to add the sensor-temperature check, and to extend the runs to days with plants in the enclosure. The logged data, firmware and controller code are kept with the project so the results can be reproduced.

ACKNOWLEDGMENT

The authors used an AI assistant (Anthropic Claude) to help write the laboratory firmware, the logging and analysis scripts and the controller code, and to draft parts of this text from the recorded data and the authors' notes. The experiments were carried out by the authors; all measurements reported are from the authors' hardware, and the authors take full responsibility for the content.

REFERENCES

- [1] M. Codeluppi, L. Davoli, and G. Ferrari, "Forecasting air temperature on edge devices with embedded AI," *Sensors*, vol. 21, no. 12, Art. no. 3973, 2021.
- [2] I. Ihoume, R. Tadili, N. Arbaoui, M. Benchirfa, A. Idrissi, and M. Daoudi, "Developing a multi-label tinyML machine learning model for an active and optimized greenhouse microclimate control from multivariate sensed data," *Artif. Intell. Agric.*, vol. 6, pp. 129–137, 2022.
- [3] K. A. D. Sukowati et al., "K-NN based TinyML on ESP32 for mushroom cultivation climate control," *J. Nas. Tek. Elektro Teknol. Inf. (JNTETI)*, vol. 15, no. 3, 2026, doi: 10.22146/jnteti.v15i3.25532.
- [4] H. Ren, D. Anicic, and T. A. Runkler, "TinyOL: TinyML with online-learning on microcontrollers," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021.
- [5] S. Disabato and M. Roveri, "Tiny machine learning for concept drift," *IEEE Trans. Neural Netw. Learn. Syst.*, 2022, doi: 10.1109/TNNLS.2022.3229897.
- [6] M. Pavan, E. Ostrovan, A. Caltabiano, and M. Roveri, "TyBox: An automatic design and code generation toolbox for TinyML incremental on-device learning," *ACM Trans. Embed. Comput. Syst.*, 2023.
- [7] B. Morcego, W. Yin, S. Boersma, E. van Henten, V. Puig, and C. Sun, "Reinforcement learning versus model predictive control on greenhouse climate control," *Comput. Electron. Agric.*, vol. 215, Art. no. 108372, 2023.

- [8] H. Sun et al., “Grey-box humidity model identified on an STM32 for humidifier scheduling,” 2023 (as indexed by Consensus; full citation to be verified before submission).
- [9] P. Bicski et al., “A simple autoregressive model versus an autoencoder for anomaly detection on industrial time series,” IEEE Internet Things Mag., 2023 (full citation to be verified before submission).
- [10] Bosch Sensortec, “BME280: Combined humidity and pressure sensor,” Datasheet BST-BME280-DS002, Reutlingen, Germany.
- [11] Espressif Systems, “ESP32 series datasheet,” Shanghai, China. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>